

P, NP, NP Hard, NP Complete

- Focus of this lecture = get the definitions across

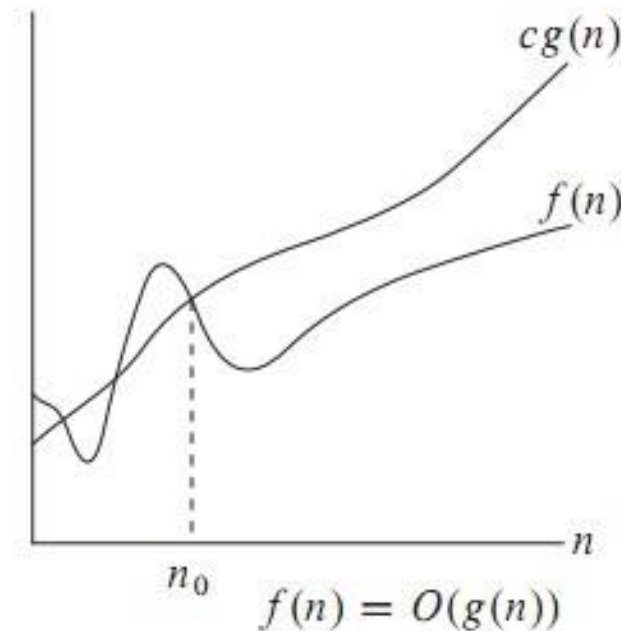
Complexity definitions

- Big-Oh
- Big-Theta
- Big - Omega

Big Oh (O)

$f(n) = O(g(n))$ *iff* there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$

O -notation to give an **upper bound** on a function

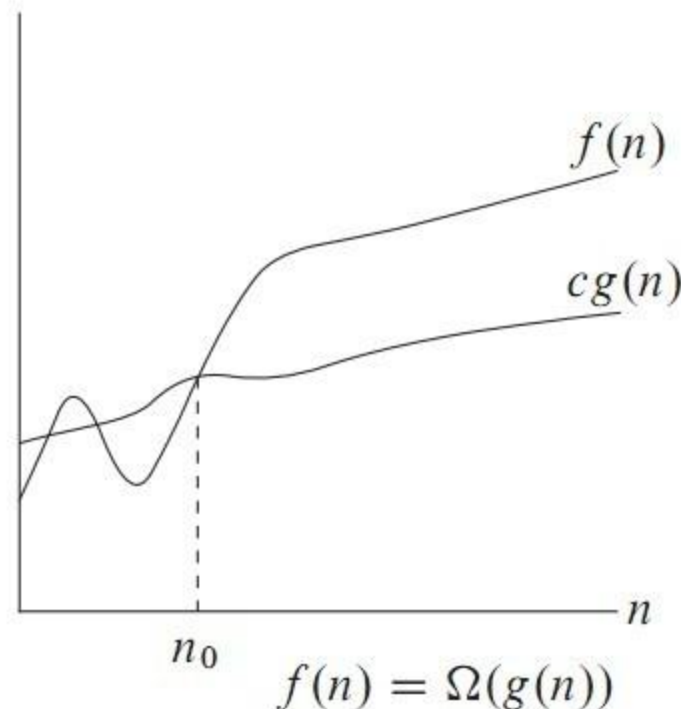


Omega Notation

Big oh provides an asymptotic **upper** bound on a function.

Omega provides an asymptotic **lower** bound on a function.

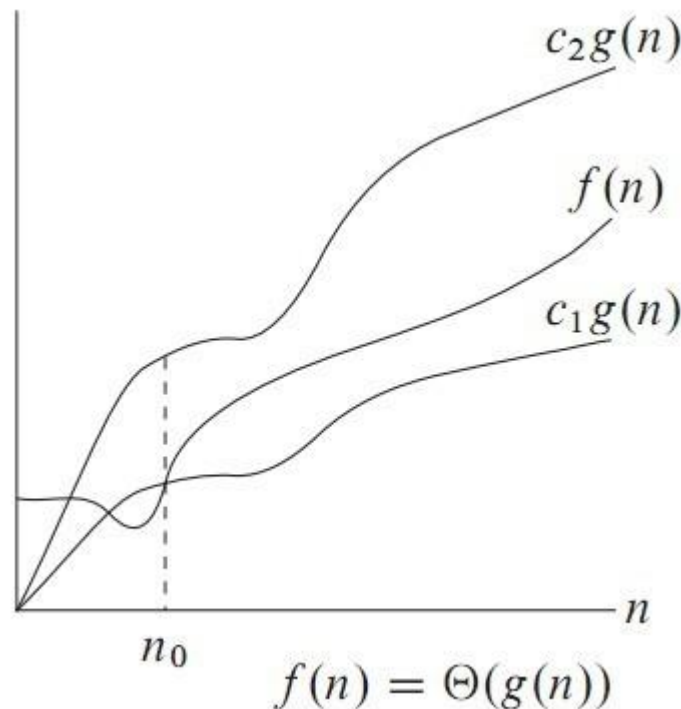
$$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}.$$



Theta Notation

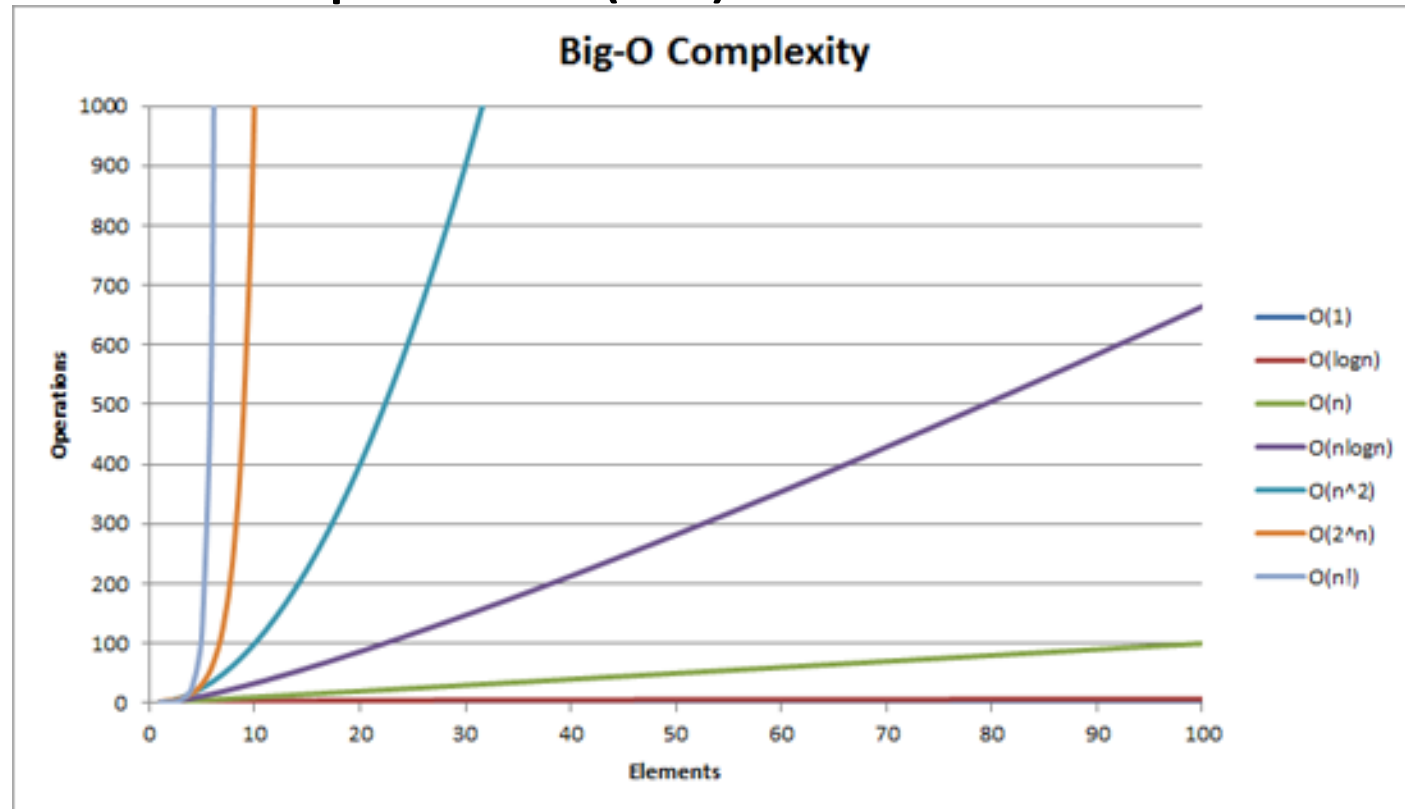
Theta notation is used when function f can be bounded **both from above and below** by the same function g

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\} .^1$



How bad is exponential complexity

- Fibonacci example – the recursive fib cannot even compute fib(50)



The class P

- The class P consists of those problems that are solvable in polynomial time.
- More specifically, they are problems that can be solved in time $O(n^k)$ for some constant k , where n is the size of the input to the problem
- The key is that n is the **size of input**

What is the complexity of primality testing?

```
public static boolean isPrime(int n){
    boolean answer = (n>1)? true: false;

    for(int i = 2; i*i <= n; ++i)
    {
        System.out.printf("%d\n", i);
        if(n%i == 0)
        {
            answer = false;
            break;
        }
    }
    return answer;
}
```

This loops until the square root of n
So this should be $O(\sqrt{n})$

But what is the input size?
How many bits does it take to
represent the number n?
 $\log(n) = k$

What is $\sqrt{n}$

$$\sqrt{n} = \sqrt{2^{\log(n)}} = (2^k)^{0.5}$$

Naïve primality testing is
exponential!!

Why obsess about primes?

- Crypto uses it heavily
- Primality testing actually is in P
- Proven in 2002
 - Uses complicated number theory
 - AKS primality test

NP

- **NP is not the same as non-polynomial complexity/running time. NP does not stand for not polynomial.**
- **NP = Non-Deterministic polynomial time**
- NP means verifiable in polynomial time
- Verifiable?
 - If we are somehow given a ‘certificate’ of a solution we can verify the legitimacy in polynomial time

What happened to automata?

- Problem is in NP iff it is decidable by some non deterministic Turing machine in polynomial time.
- Remember that the model we have used so far is a deterministic Turing machine
- It is provable that a Non Deterministic Turing Machine is equivalent to a Deterministic Turing Machine
- Remember NFA to DFA conversion?
 - Given an NFA with n states how many states does the equivalent DFA have?
 - Worst case 2^n
- The deterministic version of a poly time non deterministic Turing machine will run in exponential time (worst case)

NP problems

- Graph theory has these fascinating(annoying?) pairs of problems
 - Shortest path algorithms?
 - Longest path is NP complete (we'll define NP complete later)
 - Eulerian tours (visit every vertex but cover every edge exactly once, even degree etc). Solvable in polynomial time!
 - Hamiltonian tours (visit every vertex, no vertices can be repeated). NP complete

Hamiltonian cycles

- Determining whether a directed graph has a Hamiltonian cycle does not have a polynomial time algorithm (yet!)
- However if someone was to give you a sequence of vertices, determining whether or not that sequence forms a Hamiltonian cycle can be done in polynomial time
- Therefore Hamiltonian cycles are in NP

SAT

- A boolean formula is ***satisfiable*** if there exists some assignment of the values 0 and 1 to its variables that causes it to evaluate to 1.
- CNF – Conjunctive Normal Form. ANDing of clauses of ORs

$$(x_0 \vee x_2) \wedge (\neg x_0 \vee x_1) \wedge (x_0 \vee x_1 \vee \neg x_2)$$

2-CNF SAT

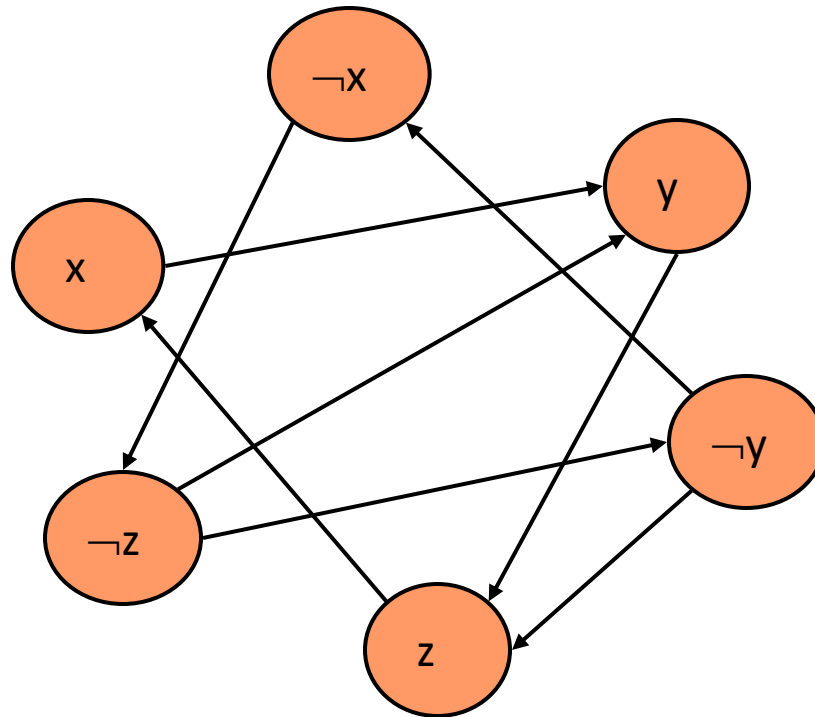
- Each or operation has two arguments that are either variables or negation of variables
- The problem in 2 CNF SAT is to find true/false(0 or 1) assignments to the variables in order to make the entire formula true.

$$(\neg x \vee y) \wedge (\neg y \vee z) \wedge (x \vee \neg z) \wedge (z \vee y)$$

- Any of the OR clauses can be converted to implication clauses

2-SAT is in P

- Create the implication graph



Satisfiability via path finding

- If there is a path from x to $\neg x$
- And if there is a path from $\neg x$ to x
- Then FAIL!
- How to find paths in graphs?
 - DFS/BFS and modifications thereof

3 CNF SAT (3 SAT)

- Not so easy anymore.
- Implication graph cannot be constructed
- No known polytime algorithm
- Is it NP?
 - If someone gives you a solution how long does it take to verify it?
 - Make one pass through the formula and check
- This is an NP problem

P is a subset of NP

- Since it takes polynomial time to run the program, just run the program and get a solution
- But is NP a subset of P?
- No one knows if $P = NP$ or not
- Solve for a million dollars!
 - <http://www.claymath.org/millennium-problems>

What is not in NP?

- Undecidable problems
 - Given a polynomial with integer coefficients, does it have integer roots
 - Hilbert's nth problem
 - Impossible to check for all the integers
 - Even a non-deterministic TM has to have a finite number of states!
 - More on decidability later
- Tautology
 - A boolean formula that is true for all possible assignments
 - Here just one 'verifier' will not work. You have to try all possible values

Reducibility

- a problem Q can be reduced to another problem Q' if any instance of Q can be “easily rephrased” as an instance of Q' , the solution to which provides a solution to the instance of Q
- Is a linear equation reducible to a quadratic equation?
 - Sure! Let coefficient of the square term be 0

NP - hard

- What are the hardest problems in NP?

$$L_1 \leq_p L_2$$

- That notation means that L_1 is reducible in polynomial time to L_2 .
- The less than symbol basically means that the time taken to solve L_1 is no worse than a polynomial factor away from the time taken to solve L_2 .

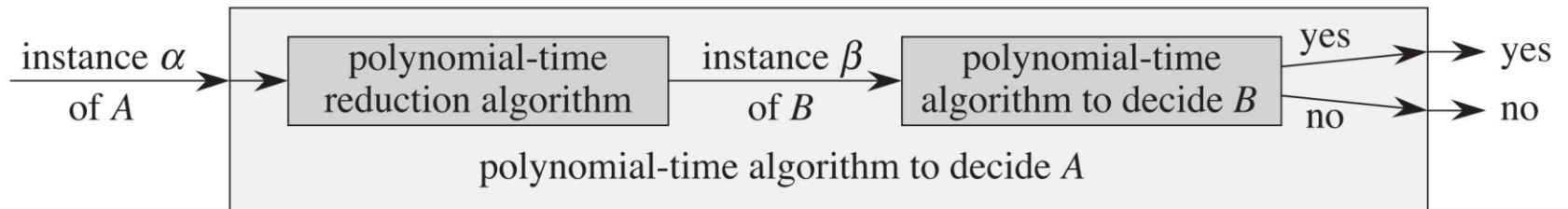
NP-hard

- A problem (a language) is said to NP-hard if every problem in NP can be poly time reduced to it.

$$L' \leq_p L \text{ for every } L' \in NP$$

NP Complete problems/languages

- Need to be in NP
- Need to be in NP-Hard
- If both are satisfied then it is an NP complete problem
- Reducibility is a transitive relation.
- If we know a single problem in NP-Complete, that helps when we are asked to prove some other problem is NP-Complete
- Assume problem P is NP Complete
All NP problems are reducible to this problem
Now given a different problem P'
If we show P reducible to P'
Then by transitivity all NP problems are reducible to P'



What is in NP-Complete

- For this course, we will axiomatically state that the following problems are NP-Complete
- SAT – Given any boolean formula, is there some assignment of values to the variables so that the formula has a true value
- 3-CNF SAT
- Actually any boolean formula can be reduced to 3-CNF form

An example reduction

- CLIQUE problem (We want to show it is NP-hard)
- A clique in an undirected graph is a subset of vertices such that each pair is connected by an edge
- We want to take a problem instance in 3-CNF SAT and convert it to CLIQUE finding

Reducing 3CNF SAT to CLIQUE

- Given – A boolean formula in 3 CNF SAT
- Goal – Produce a graph (in polynomial time) such that

Satisfiability $\Leftrightarrow$ Clique of a certain size

- We will construct a graph where satisfying formula with k clauses is equivalent to finding a k vertex clique.

Decision problems versus optimization problems

- Finding the maximum sized clique is an optimization problem
- But we can reduce it to a series of decision problems
 - Can we find a clique of size 3 (why start at 3??)
 - Can we find a clique of size 4
 - Etc
- In general in our study of NP etc, we will focus on decision problems

CLIQUE

- An instance of a clique problem gives you 2 things as input
 - Graph
 - Some positive integer k
- Question being asked = do we have a clique of size k in this graph
- Why can't I just go through and pick all possible k -subsets?

REDUCE 3-CNF SAT to CLIQUE

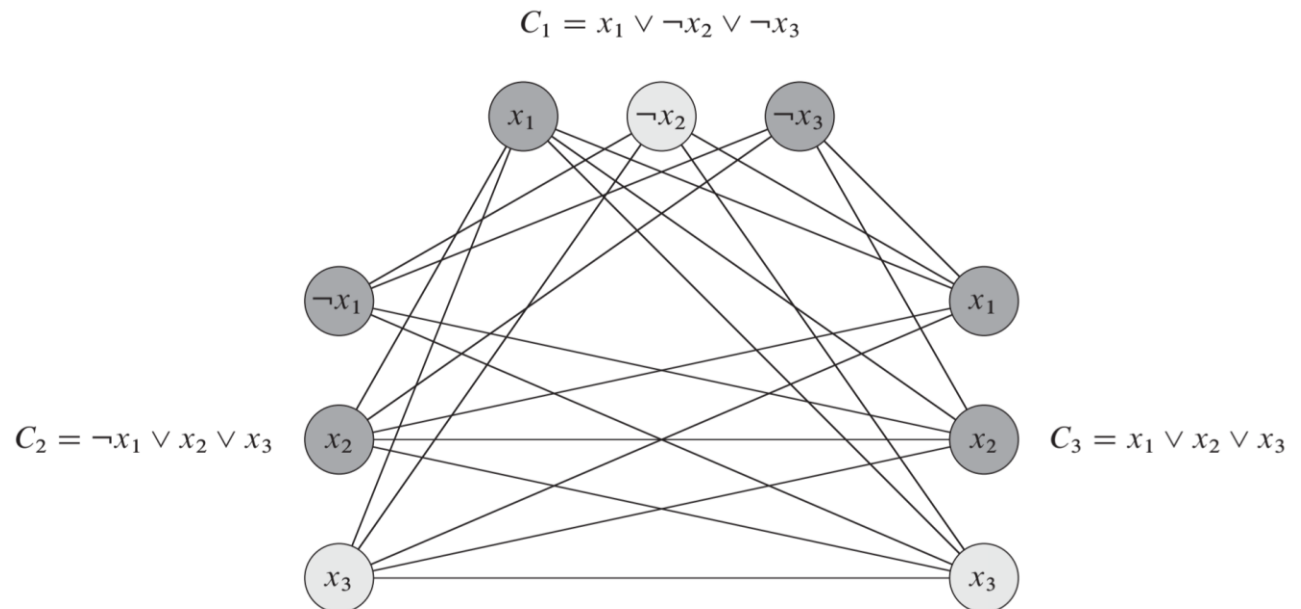
$$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

For each clause, create a vertex for each literal

For the edges

Connect vertices if they come from different clauses

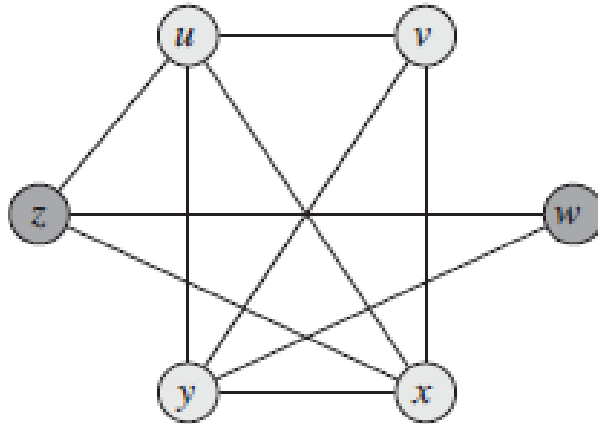
Even if the vertices come from different clauses, do not connect if it results in incompatibility. No variable should be connected to its not.



Vertex cover problem

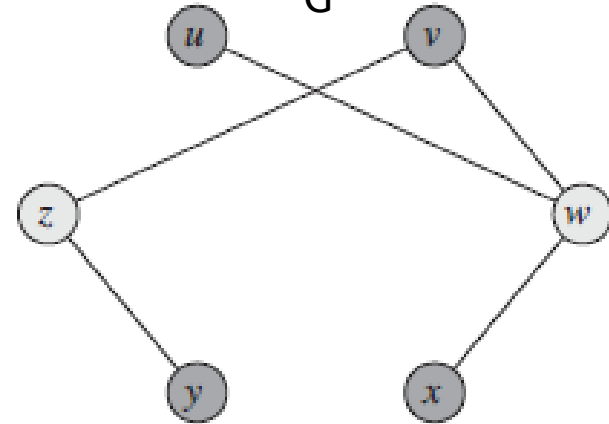
- A vertex cover of an undirected graph $G=(V,E)$ is a subset of vertices such that every edge is incident to at least one of the vertices
- We're typically interested in finding the minimum sized vertex cover
- To show vertex cover is NP-complete
- What problem should we try to reduce to it
- It sounds like the 'reverse' of CLIQUE
- Reduction is done from CLIQUE to vertex cover

G



(a)

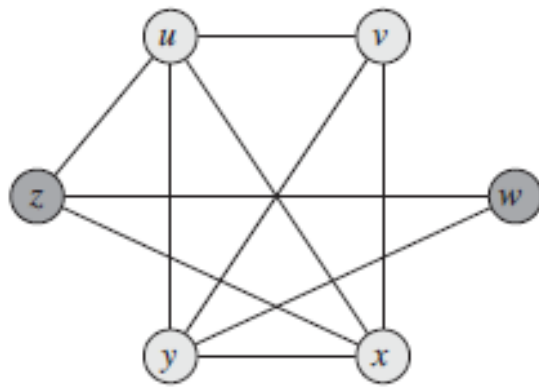
$G' =$
Complement
Of
G



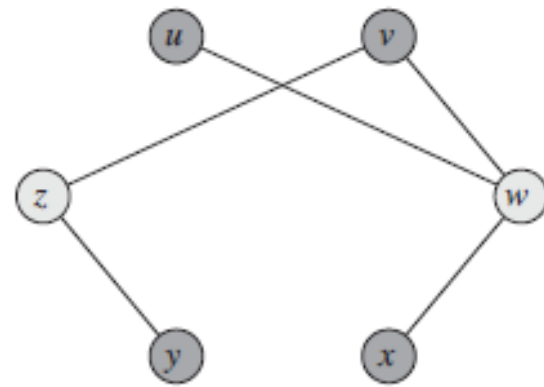
(b)

Clique of size k in G exists iff a vertex cover of size $|V| - k$ exists in G' where G' is the complement graph (vertices that had an edge between them in G do not have one in G' and vice versa)

$$\text{CLIQUE} \leq_p \text{VC}$$



(a)



(b)

The original graph has a u, v, x, y CLIQUE. That is a clique of size 4

The complement graph has a vertex cover of size 6 (number of vertices) – 4 (clique size). z, w is one such vertex cover.